

Reviewer Pack — Minimal Norm of Hodge Classes in Kähler Geometry and Circuit...

Entry 74374bb75e90 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 00:21:41 UTC

Minimal Norm of Hodge Classes in Kähler Geometry and Circuit Monotone Width Correlation

Entry ID: 74374bb75e90 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 00:21:41 UTC

1. Conjecture

Field A (mathematical branch): Kähler Geometry **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For every n -variable Boolean circuit C with monotone width $w(C)$, the minimal norm of a generic Hodge class in its associated Kähler manifold is linearly correlated with its monotone width, such that $\min_norm(H) = \Theta(w(C))$ for any generic Hodge class H .

Rationale (proposer's reasoning):

Kähler geometry offers a rich structure to encode geometric properties that might correlate with the complexity of Boolean functions. If such correlations exist, they could provide new insights into the intrinsic difficulty of computational problems.

Taxonomy category: KÄHLER_GEOMETRY_CIRCUIT_MONOTONE_WIDTH (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a14887e091ac6bef

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given Boolean circuit C , if the Pearson correlation coefficient between $\min_norm(H)$ and $w(C)$ is ≥ 0.8 for all generic Hodge classes H , then support is provided for the conjecture.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Kähler geometry" AND "Boolean circuit complexity" AND "Hodge class"
- "monotone width" INtitle "Kähler geometry" AND "circuit monotone width" - "minimal norm
Hodge classes" AND "correlation with Boolean circuits"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.3s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_monotone_circuit(n):
        # Generate a random monotone circuit with n variables
        circuit = []
        for i in range(1, n):
            gate = random.choice(['AND', 'OR'])
            inputs = [random.randint(0, 1) for _ in range(i)]
            circuit.append((gate, inputs))
        return circuit

    def compute_monotone_width(circuit):
        # Compute the monotone width of the circuit
        width = 0
        for gate, inputs in circuit:
            width = max(width, len(inputs))
        return width

    def compute_hodge_norm(n):
        # Compute a generic Hodge norm (simplified example)
        return random.uniform(1, n)

    min_norm_sum = 0
    monotone_width_sum = 0
    instances_tested = 0
    n_max = 0

    for n in [5, 10, 15, 20, 30, 40]:
        if n > n_max:
            n_max = n
```

```

    for _ in range(5): # Ensure at least 30 instances per seed
        circuit = generate_monotone_circuit(n)
        monotone_width = compute_monotone_width(circuit)
        hodge_norm = compute_hodge_norm(n)

        min_norm_sum += hodge_norm
        monotone_width_sum += monotone_width
        instances_tested += 1

if instances_tested == 0:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "No instances generated"
    }

mean_min_norm = min_norm_sum / instances_tested
mean_monotone_width = monotone_width_sum / instances_tested

if math.isclose(mean_min_norm, 0) or math.isclose(mean_monotone_width, 0):
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "Mean values are zero"
    }

correlation_coefficient = (min_norm_sum * monotone_width_sum - instances_tested * mean_min_norm * mean_monotone_width) /
    math.sqrt((min_norm_sum**2 - instances_tested * mean_min_norm**2) *
              (monotone_width_sum**2 - instances_tested * mean_monotone_width**2))

return {
    "metric_name": "Pearson correlation coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": abs(correlation_coefficient) >= 0.8,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

if all("conjecture_holds" in r and r["conjecture_holds"] for r in results):
    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = 1.0
else:
    support_fraction = sum(1 for r in results if "conjecture_holds" in r and r["conjecture_holds"]) / len(results)
    first_failing_seed = next((r["seed"] for r in results if not ("conjecture_holds" in r and r["conjecture_holds"])), None)
    counterexample = "first failing seed"

print(f"RESULT: {'SUPPORTED' if support_fraction == 1.0 else 'FALSIFIED'} mean={mean_value} std={std_dev}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

conjecture_holds': True, 'counterexample': ''
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': 0.9999999999999999, 'instances_tested': 3}
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': 0.9999999999999998, 'instances_tested': 3}
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': 1.0, 'instances_tested': 3}
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': 0.9999999999999998, 'instances_tested': 3}
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': 1.0000000000000002, 'instances_tested': 3}
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': 1.0000000000000002, 'instances_tested': 3}
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': 1.0, 'instances_tested': 3}
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': 1.0, 'instances_tested': 3}
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': 1.0, 'instances_tested': 3}
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': 1.0, 'instances_tested': 3}
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': 1.0, 'instances_tested': 3}
RESULT: SUPPORTED mean=1.0 std=9.721057689603553e-17 support_fraction=1.0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test only considers a small range of n (5 to 40) and uses a fixed number of instances per seed (5). This may not be representative of the conjecture's validity for all circuit sizes. Additionally, the 'compute_hodge_norm' function is a placeholder that returns random values, which does not accurately measure the minimal norm of Hodge classes in Kähler geometry.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge | original judge: The test results show a Pearson correlation coefficient of 1.0 for all trials, which meets the pre-registered support condition with a high degree of | next: Further investigation is needed to validate the conjecture across a wider range of circuit sizes and to refine the 'compute_hodge_norm' function to accurately measure minimal norms in Kähler geometry.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14461
2	propose	ollama_remote	glm4:latest	0	0	12857
3	preregistration	ollama_remote	glm4:latest	0	0	22087
4	novelty	ollama_remote	glm4:latest	0	0	8414
5	novelty	ollama_remote	glm4:latest	0	0	26554
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25075
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11401
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	62417
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44167
10	critic	ollama_remote	glm4:latest	0	0	38253
11	judge	ollama_remote	glm4:latest	0	0	9629

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 275313 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/74374bb75e90.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/74374bb75e90.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/74374bb75e90.tar.gz` (if generated)